

Отчет о лабораторной работе №1

**«Ознакомление со структурой Java и
работой пакетов в ней»**

Выполнил:
Петров Евгений Станиславович
Группа: 6204-010302D

Задание №1

При запуске утилит **javac** и **java** без аргументов в командной строке отображается краткая инструкция по применению со списком поддерживаемых параметров.

```
D:\java_lab\lab1\MyJar>java
Usage: java [java options...] <application> [application arguments...]

Where <application> is one of:
  <mainclass>          to execute the main method of a compiled main class
  -jar <jarfile>.jar    to execute the main class of a JAR archive
  -m <module>[/<mainclass>] to execute the main class of a module
  <sourcefile>.java     to compile and execute a source-file program

Where key java options include:
  --class-path <class path>
    where <class path> is a list of directories and JAR archives to search for class files, separated by ";"
  --module-path <module path>
    where <module path> is a list of directories and JAR archives to search for modules, separated by ";"
  -version
    to print product version to the error stream and exit

For additional help on usage:      java --help
For an interactive Java environment: jshell

D:\java_lab\lab1\MyJar>javac
Usage: javac <options> <source files>
Where possible options include:
  @<filename>          Read options and filenames from file
  -Akey[=value]         Options to pass to annotation processors
  --add-modules <module>(<module>)*
    Root modules to resolve in addition to the initial modules,
    or all modules on the module path if <module> is ALL-MODULE-PATH.
  --boot-class-path <path>, -bootclasspath <path>
    Override location of bootstrap class files
  --class-path <path>, -classpath <path>, -cp <path>
    Specify where to find user class files and annotation processors
  -d <directory>        Specify where to place generated class files
  -deprecation
    Output source locations where deprecated APIs are used
  --enable-preview
    Enable preview language features.
    To be used in conjunction with either -source or --release.
  -encoding <encoding>  Specify character encoding used by source files
  -endorseddirs <dirs>  Override location of endorsed standards path
  -extdirs <dirs>       Override location of installed extensions
  -g                    Generate all debugging info
  -g:{lines,vars,source} Generate only some debugging info
  -g:none               Generate no debugging info
  -h <directory>        Specify where to place generated native header files
  --help, -help, -?     Print this help message
  --help-extra, -X      Print help on extra options
  -implicit:{none,class}
    Specify whether to generate class files for implicitly referenced files
  -J<flag>              Pass <flag> directly to the runtime system
  --limit-modules <module>(<module>)*
    Limit the universe of observable modules
  --module <module>(<module>)*, -m <module>(<module>)*
    Compile only the specified module(s), check timestamps
  --module-path <path>, -p <path>
    Specify where to find application modules
  --module-source-path <module-source-path>
    Specify where to find input source files for multiple modules
  --module-version <version>
    Specify version of modules that are being compiled
  -nowarn
    Generate only mandatory warnings
  -parameters
    Generate metadata for reflection on method parameters
  -proc:{none,only,full}
    Control whether annotation processing and/or compilation is done.
  -processor <class1>[,<class2>,<class3>...]
    Names of the annotation processors to run;
    bypasses default discovery process
  --processor-module-path <path>
    Specify a module path where to find annotation processors
  --processor-path <path>, -processorpath <path>
    Specify where to find annotation processors
  -profile <profile>
    Check that API used is available in the specified profile.
    This option is deprecated and may be removed in a future release.
  --release <release>
    Compile for the specified Java SE release.
    Supported releases:
      8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25
  -s <directory>        Specify where to place generated source files
  --source <release>, -source <release>
    Provide source compatibility with the specified Java SE release.
    Supported releases:
```

Задание №2

Был создан файл **MyFirstProgram.java**. Его компиляция командой **javac** прошла успешно и создала файл **MyFirstClass.class**. Однако запуск программы командой **java** завершился ошибкой из-за отсутствия в классе основного метода **main**. После его добавления возникла новая ошибка, связанная с отсутствием модификатора **static**. После исправления и объявления метода как **public static void main** программа была скомпилирована и запущена, выведя в консоль сообщение «Hello World!!!». В ходе работы были изучены этапы компиляции и запуска Java-приложений синтаксис команд и правильное объявление главного метода.

```
C:\Users\User>d:

D:\>java_lab\lab1
"java_lab\lab1" не является внутренней или внешней
командой, исполняемой программой или пакетным файлом.

D:\>cd java_lab\lab1

D:\java_lab\lab1>mkdir Task2

D:\java_lab\lab1>cd Task2

D:\java_lab\lab1\Task2>notepad MyFirstProgram.java

D:\java_lab\lab1\Task2>javac MyFirstProgram.java

D:\java_lab\lab1\Task2>
D:\java_lab\lab1\Task2>java MyFirstClass
Hello world!!!
```

Задание №3

Исходный код в файле **MyFirstProgram.java** был изменен: вместо вывода строки метод **main** теперь содержит цикл для перебора и вывода в консоль всех переданных программе аргументов командной строки. После компиляции программа была запущена с набором аргументов.

```
D:\java_lab\lab1\Task2>cd D:\java_lab\lab1
D:\java_lab\lab1>mkdir Task3
D:\java_lab\lab1>cd Task3
D:\java_lab\lab1\Task3>notepad MyFirstProgram.java
D:\java_lab\lab1\Task3>cd D:\java_lab\lab1
D:\java_lab\lab1>cd Task3
D:\java_lab\lab1\Task3>javac MyFirstProgram.java
D:\java_lab\lab1\Task3>java MyFirstClass
D:\java_lab\lab1\Task3>java MyFirstClass arg1 arg2 arg3 arg4 arg5
arg1
arg2
arg3
arg4
arg5
```

Задание №4

Исходный код программы был модифицирован: добавлен новый класс **MySecondClass**. Его структура включает два приватных поля типа **int**, методы доступа (get/set), конструктор и метод **multiply()** для умножения чисел. Код в методе **main** класса **MyFirstClass** был заменен на логику, создающую и использующую объект добавленного класса. После внесения изменений программа была успешно скомпилирована и запущена.

```
D:\java_lab\lab1\Task4>javac MyFirstProgram.java
```

```
D:\java_lab\lab1\Task4>java MyFirstClass
```

```
1 2 3 4 5 6 7 8
2 4 6 8 10 12 14 16
3 6 9 12 15 18 21 24
4 8 12 16 20 24 28 32
5 10 15 20 25 30 35 40
6 12 18 24 30 36 42 48
7 14 21 28 35 42 49 56
8 16 24 32 40 48 56 64
```

```
D:\java_lab\lab1\Task4>cd D:\java_lab\lab1
```

Задание №5

В рамках задания были удалены все ранее сгенерированные файлы байт-кода (.class). Затем была создана директория **myfirstpackage**, внутри которой размещен файл **MyFirstPackage.java**. Код класса **MySecondClass** был перемещен в этот файл и удален из основного файла программы. Для обеспечения доступа в код **MyFirstProgram.java** добавлена директива **import myfirstpackage.*;**.

На этапе компиляции возникли ошибки, которые были устранены последовательным выполнением следующих действий:

1. Объявление пакета с помощью **package myfirstpackage;** в файле класса.
2. Приведение имени файла в соответствие с именем класса (**MySecondClass.java**).
3. Объявление класса **MySecondClass** как **public**.

```
D:\java_lab\lab1>mkdir Task5

D:\java_lab\lab1>cd Task5

D:\java_lab\lab1\Task5>mkdir myfirstpackage

D:\java_lab\lab1\Task5>notepad myfirstpackage\MySecondClass.java
```

```
D:\java_lab\lab1\Task5>notepad MyFirstProgram.java

D:\java_lab\lab1\Task5>javac -d . myfirstpackage\MySecondClass.java MyFirstProgram.java

D:\java_lab\lab1\Task5>java MyFirstClass
1 2 3 4 5 6 7 8
2 4 6 8 10 12 14 16
3 6 9 12 15 18 21 24
4 8 12 16 20 24 28 32
5 10 15 20 25 30 35 40
6 12 18 24 30 36 42 48
7 14 21 28 35 42 49 56
8 16 24 32 40 48 56 64

D:\java_lab\lab1\Task5>cd D:\java_lab\lab1
```

Задание №6

В ходе выполнения задания были изучены возможности утилиты `jar` через командную строку. Все скомпилированные `.class`-файлы из предыдущей задачи были скопированы для архивации. Был создан файл `manifest.mf`, содержащий информацию об авторе (ФИО), с обязательной пустой строкой в конце.

Затем был собран JAR-архив, включающий созданный манифест и файлы классов. Для проверки архива была создана отдельная директория `MyJar`, куда он был перемещен и

откуда успешно запущен.

```
D:\java_lab\lab1\Task5>cd D:\java_lab\lab1

D:\java_lab\lab1>mkdir Task6

D:\java_lab\lab1>cd Task6

D:\java_lab\lab1\Task6>copy D:\java_lab\lab1\Task5\MyFirstClass.class
Скопировано файлов:      1.

D:\java_lab\lab1\Task6>mkdir myfirstpackage

D:\java_lab\lab1\Task6>copy D:\java_lab\lab1\Task5\myfirstpackage\MySecondClass.class myfirstpackage\
Скопировано файлов:      1.

D:\java_lab\lab1\Task6>notepad manifest.mf

D:\java_lab\lab1\Task6>jar cfm myfirst.jar manifest.mf MyFirstClass.class myfirstpackage\MySecondClass.class

D:\java_lab\lab1\Task6>mkdir ..\MyJar

D:\java_lab\lab1\Task6>move myfirst.jar ..\MyJar\
Перемещено файлов:      1.

D:\java_lab\lab1\Task6>cd ..\MyJar

D:\java_lab\lab1\MyJar>java -jar myfirst.jar
1 2 3 4 5 6 7 8
2 4 6 8 10 12 14 16
3 6 9 12 15 18 21 24
4 8 12 16 20 24 28 32
5 10 15 20 25 30 35 40
6 12 18 24 30 36 42 48
7 14 21 28 35 42 49 56
8 16 24 32 40 48 56 64
```